AlphaZero Solves the Rubik's Cube from Uniform Random Starts

Lee Davis *davislee4891@gmail.com*
The Ohio State University

Abstract

Prior learned solvers of the 3×3×3 Rubik's cube train on goal-anchored data, generating their training states backward from the solved cube. We show this is unnecessary. A standard AlphaZero-style pipeline, trained only on uniformly random cubes and rewarded only for fractional facelet completion and for solving, solves all 1,000 cubes of DeepCubeA's benchmark on all three seeds. Apart from a symmetry augmentation, it changes AlphaZero in one place, a value target in which every label is a peak the episode itself witnessed. Matched-compute ablations show that the target and the augmentation are both needed. Its solutions are not optimal (54.4 quarter turns, against 21.5 for DeepCubeA), but it solves consistently. It does so using only a nearly uninformative reward whose ranking inverts near the goal, where hundreds of local maxima outrank every state one move from solved. Without search, its policy head solves 78.8% of the benchmark with one network call per move. We release a compact implementation that trains in four hours on one V100.

1 Introduction

The 3×3×3 Rubik's cube is a canonical hard-exploration benchmark, with 4.33×10^{19} states, a single goal state, and a diameter of 20 moves (Rokicki et al., 2014). The difficulty lies in guiding search to the goal. Solutions are short, but the state space is so large that random exploration almost never finds the goal.

Prior learned solvers of the full cube avoid this problem by training on goal-anchored data, generating their training states backward from the solved cube (Section 2). We ask whether that is necessary. Our agent sees only uniformly random cubes and observes exactly two signals, the fraction of facelets showing their solved color and whether the cube is solved. The reward is the naive progress measure, which we defined once and never tuned to guide search. It is dense but provides little useful guidance. Over most of the state space it carries almost no information about true distance (fitness–distance correlation 0.055 on uniform cubes). Where it does carry information, its ranking inverts, and certain states six moves from solved outrank every state one move away. Exploiting it without learning is correspondingly expensive. The cheapest procedure we found that solves at least one uniformly random cube spends 2.0×10^6 node expansions (Section 3.2).

We find that goal-anchored data is not necessary. A standard AlphaZero-style pipeline (Silver et al., 2018) trained in this setting bootstraps from 0% to a complete solve of the standard DeepCubeA 1,000-cube test set on all three seeds. To our knowledge it is the first learned solver of the full 3×3×3 to train on only uniformly random cubes. It adds no new architecture, search method, or auxiliary loss to standard AlphaZero. Apart from a symmetry augmentation, the recipe changes AlphaZero in one place. It uses a value target in which every label is a peak that the episode actually reached (Section 4.2), and matched-compute ablations show that the recipe depends on this change (Section 5). The trained agent converges on a classical decomposition of the problem. Its solves end with a long run of moves inside a domino group, a subgroup reachable using only ten of the cube's eighteen moves. Thistlethwaite's and Kociemba's algorithms use the same subgroup as an intermediate stage. The agent enters it about 17 moves before the solve, against about 10 for an unlearned search on the same reward (Section 6).

Table 1: Prior learned cube solvers, organized by where the goal enters training. Solve rates and lengths follow each paper's own protocol and test set. Ours is DeepCubeA's 1,000-cube set. All lengths are in the quarter-turn metric (QTM). "1-ply greedy" evaluates the learned distance at every neighbour and steps to the minimum. "Policy argmax" takes the top move of the policy head in one network call. †Three-seed mean at 1,024 eval-time simulations under the protocol of Section 5.

Method	Training states	Supervision	Search	Solved	Length
DeepCube	reverse scrambles	bootstrapped value	MCTS	640/640	median 30
DeepCubeA	reverse scrambles	cost-to-go	weighted A*	100%	mean 21.50
EfficientCube	reverse scrambles	last-move prediction	beam search	100%	mean 21.26
CayleyPy	goal-anchored walks	diffusion distance	beam search	100%	98.4% opt.
Lin & Liang	forward walks	learned pairwise cost	policy rollout	99.4% (2×2×2)	—
CRTR	reverse scrambles	contrastive distance	1-ply greedy	100%	mean 448.7
Ours	uniform random starts	completion r + solve bit	PUCT search	100%†	mean 54.4
Ours, no search	uniform random starts	completion r + solve bit	policy argmax	78.8% (61.6–88.2)	mean 80.6

2 Related Work

Table 1 organizes prior learned solvers by where the solved state enters their training data.

Goal-anchored training data. DeepCube (McAleer et al., 2018; 2019) and DeepCubeA (Agostinelli et al., 2019) generate training states by scrambling outward from the solved cube, bootstrap a cost-to-go function by value iteration, and solve with heavy test-time search. Takano (2023) learns to invert scrambles directly. The CayleyPy project (Chervov et al., 2025a;b) scales the same idea to Cayley graphs of up to 10^{70} nodes, reporting the first learned results on the 4×4×4 and 5×5×5. Apart from Lin & Liang (2024), discussed below, every smaller learned system we are aware of generates its training states from the solved cube in the same way (Rute, 2018; Chasmai, 2021; Corli et al., 2021; Poesia et al., 2021; Konen, 2023; Barro, 2025), or regresses a classical solver's distance labels (Brunetto & Trunda, 2017). All place the goal inside the training distribution by construction. This line of work is complementary to ours. It optimizes solution quality and scale given goal-anchored data, whereas we ask whether a general method can solve the cube reliably without that data. Ziarko et al. (2025) report the first method to solve arbitrary cube states without any search. It uses greedy descent on a contrastive temporal distance learned from depth-21 reverse scrambles that terminate at the solved cube, and it solves 100% of 1,000 instances at mean length 448.7 within a 6,000-move budget. Our search-free solutions are 5.6× shorter under a budget at least 37× tighter in quarter turns, at a lower search-free solve rate. Their descent excludes visited states. Our argmax is deterministic given a one-step face memory, so the cubes it misses end in cycles that no move budget resolves (Section 6). Lin & Liang (2024) avoid goal-anchored data by learning a cost between arbitrary pairs of states. They work on the 2×2×2 cube, whose $\sim 3.7 \times 10^6$ states can be enumerated exhaustively.

Single-agent AlphaZero and witnessed-peak value targets. By "standard pipeline" we mean AlphaZero (Silver et al., 2018) specialized to the single-agent case (Silver et al., 2017; Schrittwieser et al., 2020). Two-player self-play supplies an automatic curriculum, since the opponent is always exactly as strong as the learner. A single-agent puzzle with a fixed start distribution has no analogous mechanism. Laterre et al. (2018) create one by ranking each episode's return against a percentile of recent returns, so the label attached to a fixed episode changes as the buffer does. Our label is instead taken from a peak the episode reached, and it does not change during training. Regressing a target higher than the realized return is an established approach. Episodic control (Blundell et al., 2016; Lin et al., 2018) stores for each state the highest return ever observed from it. Our maximum propagates in a different direction, as Section 4.2 discusses. Max-reward reinforcement learning (Gottipati et al., 2020; Veviurko et al., 2024) optimizes the maximum reward along a trajectory instead of the sum, and learns it with Bellman-style backups. Our value target uses no such backup and regresses each episode's peak onto all of its states. The two standard remedies from the hard-exploration literature relate to our setting in different ways. A curriculum (Bengio et al., 2009) orders instances by difficulty. On the cube, the easy instances are cubes scrambled a few moves from solved, so building a curriculum means generating states backward from the goal. We do not use one, and our train-

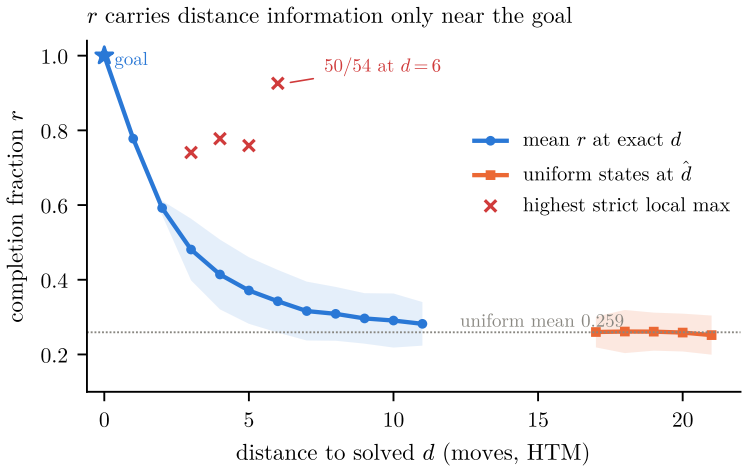


Figure 1: The completion reward as a guidance signal. Mean completion r against distance to solved (full enumeration for $d \leq 6$, sampled exact distances for $d \leq 11$, two-phase estimates $\hat{d}$ beyond; bands ± 1 sd). The stratum means fall monotonically out to d=11 and are flat beyond, while the overlapping bands show that per-state discrimination is confined to $\sim$5 moves. Red crosses mark the highest strict local maximum of r at each distance.

ing distribution equals our evaluation distribution from the first episode. Go-Explore (Ecoffet et al., 2021), which archives promising states and returns to them, needs no goal-anchored data. It adds components to the pipeline, and we study what the pipeline does without such additions.

3 Setting: The Cube as a Hard-Exploration MDP

3.1 Markov decision process

States are facelet configurations of the 3×3×3 cube, and actions are the 18 half-turn-metric face turns. The reward at every step is the fraction of facelets already showing their solved color. The solve event is the only other signal the agent receives (Section 3.2 characterizes this reward in full). Each episode starts at an independent uniformly random solvable configuration, sampled in closed form over the cube group, so all reachable states are equally probable. Episodes run for at most 40 moves (2× the group diameter of 20 (Rokicki et al., 2014)), terminating early only on a solve.

3.2 Why the setting is nontrivial

The only dense signal is the completion fraction $r(s) \in [0, 1]$: the fraction of the 54 facelets showing their solved color. The six center facelets never move, so $r \geq 6/54$. A uniformly random cube scores $r = 0.259 \pm 0.052$ (mean $\pm$ sd), and one face turn changes r by at most 12/54 (Appendix D). The solve predicate adds nothing beyond $r(s) = 1$.

However, this density does not make r a useful guide. We characterize r against ground-truth distance-to-solve: exact for $d \leq 11$, a symmetry-minimized two-phase upper bound $\hat{d}$ beyond (protocol and validation in Appendix D). Figure 1 shows the fitness–distance relationship (Jones & Forrest, 1995). Averaged over a stratum, r falls monotonically with true distance, from 0.778 at d=1 to 0.282 at d=11, and is flat beyond, 0.252–0.261 across every $\hat{d} \in [17, 21]$. Per state, the discrimination behind that trend is confined to states near the goal, with $r_{\mathrm{FDC}} = 0.87$ within $d \leq 5$, against 0.14 within $d \in [6, 8]$, 0.09 within $d \in [9, 11]$, and 0.055 across uniformly random cubes. Under the standard reading of fitness–distance correlation ($|r_{\mathrm{FDC}}| < 0.15$: uncorrelated, "difficult"), the reward is uninformative over almost the entire region a forward-trained agent occupies. The state-level correlation near the goal does not carry over to individual moves. The steepest-

completion move reduces true distance only 38% of the time at d=4 and 28% at d=5, so greedy ascent fails even in the region where r_{FDC} is high.

Near the goal, where r does track distance on average, its ranking contains many local maxima. Full enumeration of the radius-6 ball yields 16,246 strict local maxima of r, and 330 of them outrank every goal-adjacent state. Appendix D also reports the reward's autocorrelation, neutrality, and plateau structure.

We use beam search to measure how much search it takes to solve cubes from r with no learning. Beam search is the search method that two of the goal-anchored solvers use (Takano, 2023; Chervov et al., 2025a). We run it on r alone over the first 50 cubes of the same DeepCubeA test set (Table 6 and Figure 4, protocol in Appendix D). It needs 10^7 to 10^9 expansions. Below 10^6 the beam solves no cubes, its first solve of any cube comes at 2.0×10^6, and 100% takes 9.2×10^8.

In comparison, the trained policy head spends 55 node expansions per cube, one network call per move with no tree and no value function, and solves 78.8% of the same benchmark (Section 5). The beam needs 2.0×10^8 to reach that rate, 3.6×10^6 times more. One self-play episode spends 1.6×10^4, which is 125 times less than the beam's first solve of any cube. Climbing the reward within that budget therefore cannot account for the trained agent's solve rate. A 50-cube sample cannot reliably detect a solve rate as low as that of early self-play (Section 5), so this comparison does not show where the first self-play solves come from. These ratios count node expansions, not compute. A network forward takes about 1.5×10^7 operations, and a beam expansion takes about 810 (Appendix D). We use the beam to characterize the reward, not to rank it against the trained agent.

4 Method

Our pipeline is AlphaZero (Silver et al., 2018) reduced to the single-agent case. Table 2 enumerates every departure from the standard recipe, including the training targets below.

Table 2: Everything cube-specific in the pipeline and every deviation from the standard AlphaZero recipe. Section 5 ablates each training deviation and the symmetry augmentation. Nothing below supplies the goal's location or any distance to it that the agent did not observe. The two distance-carrying targets encode only distances observed within the same episode (Section 4.2).

Ingredient	What it does	Status
Redundancy cut	search never re-turns the just-turned face (branching $18 \to 15$)	standard in cube search; goal-free
Symmetry augmentation	each replay draw applies one of the cube's 48 symmetries (Section 4.3)	generic invariance prior (AlphaGo Zero augmented with Go's 8 board symmetries, and Rute (2018) with the cube's 48)
Episode maximum	unsolved states regress the episode's peak completion rather than the terminal outcome	training deviation
End discounting	a solve's value decays with the observed distance to it, so the state m moves before a solve regresses $2 - 0.02\,m$	training deviation
Solve margin	solved episodes regress WIN_VALUE $= 2$, double the reward ceiling of 1	training deviation
Greedy root action	the played move is always the visit-count argmax, so exploration comes only from Dirichlet root noise	pipeline configuration
Proven-win override	a root child that is the solved state gets a one-hot visit distribution, which self-play also records as the policy target (Appendix F)	pipeline configuration

4.1 Search and network

Moves are chosen by standard PUCT (Rosin, 2011) with $c_{\text{puct}} = 1$ and 400 simulations, and the played action is the root visit-count argmax. Self-play mixes Dirichlet noise into the root prior ($\alpha = 0.3$, $\varepsilon = 0.25$), and evaluation omits it. The leaf value is the network's scalar output, used unchanged, except that a leaf that is the solved state is scored WIN_VALUE. The trunk is the DeepCubeA residual architecture (Agostinelli et al., 2019), chosen so that capacity is comparable to goal-anchored baselines. It takes the 54 facelets one-hot encoded over 6 colors (324 inputs), followed by a 5000-unit and a 1000-unit fully connected layer and then four 1000-unit residual blocks, all batch-normalized. Two linear heads follow: policy logits over the 18 moves, and a single scalar value.

4.2 Learning

Each self-play state stores the normalized root visit counts π and a value target z. In a solved episode, the state visited m moves before the solve regresses the per-move-decayed solve value $z = \text{WIN_VALUE} - 0.02\,m$ with WIN_VALUE $= 2$. Every state of an unsolved episode regresses $z = \max_t r(s_t)$, the *episode-maximum* completion. The loss is the standard two-term AlphaZero objective, cross-entropy from $p_\theta(s)$ to π plus squared error from $v_\theta(s)$ to z, optimized with Adam and no gradient clipping. v_θ therefore approximates the expected witnessed peak of an episode passing through s, rather than a return or V^π. For a fixed state it mixes the two branches in whatever proportion the current policy solves from it. The branches also have different ranges, $[0, 1]$ for the unsolved branch against $[1.2, 2]$ for the solved one. Once solves are frequent, the solved branch dominates the squared error, and the same scalar is used unchanged as the MCTS leaf value.

We call this single design *witnessed-peak value targets*. Every label is a peak the episode witnessed, either its best completion or its own solve, and it is propagated back to the states that witnessed it. The design has three components, the episode maximum, the solve margin, and the per-move decay. Together they make up the pipeline's entire departure from AlphaZero's terminal-outcome regression.

The episode maximum is justified by reversibility. Because cube moves are reversible, every state on the trajectory could in principle return to the exact peak the episode later reaches. Regressing that peak therefore spreads the reward's rare local peaks across whole trajectories instead of discarding them when the episode ends. We call such a peak witnessed-reachable. It is reachable because moves can be reversed, and it is witnessed because the episode already passed through it. This is the difference from episodic control that Section 2 deferred. Episodic control labels s with the best return achieved *from* s, a quantity the agent could still collect. We label s with the best completion witnessed *anywhere* in the episode, including at states reached after s. A peak that is reachable in principle is not necessarily reachable within the remaining budget. This leads to two caveats, both costs of propagating the peak backward as well as forward. The search corrects neither, since it uses v_θ as given (Appendix F).

The per-move decay and the solve margin order the labels that the episode maximum spreads. Every state of a solved episode scores at least $2 - 0.02 \cdot 40 = 1.2$ under the 40-move budget, above every non-solving return (≤ 1). States nearer the witnessed solve also outrank states farther from it.[1] Unlike the constants, the decay mechanism was added in response to a result. We added it after watching a flat-label variant peak and then regress, and the end-discounting ablation of Section 5 reproduces that collapse. The label thus encodes a distance the episode witnessed. Section 5 ablates all three choices.

4.3 Replay and loop

Each iteration plays 64 games, appends them to a replay buffer holding the last 16 self-play rounds (capacity 2×10^5 states), and takes SGD steps at batch size 512, targeting ∼48 draws per stored sample. Every draw applies an independent uniformly random element of the cube's 48-element symmetry group to the state, remapping facelets, colors, and the policy target together. The scalar value target is symmetry-invariant. Section 5 also ablates this augmentation. Appendix A lists every hyperparameter.

[1] Neither constant was swept. Each is the first value we tried, and we kept it. WIN_VALUE=2 is the first integer above the reward ceiling of 1, and the 0.02 decay is a round value small enough to keep even the most-decayed win above the ceiling.

Table 3: Matched-compute one-factor ablations after 240 minutes of from-scratch training, greedy evaluation (Section 5 protocol). The first row is the recipe at the 1,024-simulation budget this paper reports throughout. Every row below it, recipe and arms alike, is evaluated at the 400-simulation training budget, so that the one-factor comparison is run-for-run. Lengths are half-turn moves over solved cubes. Each ablation runs seed 0 and changes one factor, except that the no-solve-margin arm also drops end discounting. Its one-factor control is the no-end-discounting arm (Appendix B). The last two columns are training-time quantities: the wall-clock minute at which self-play first reaches the goal, and the number of self-play episodes that reach it over the whole run (out of 1.5–1.9×10^5).

			solution length			1st solve	solves
Configuration	Solve rate	95% CI	mean	med.	p90	(min)	seen
mainline budget (1,024 simulations per move)							
final recipe, 3 seeds	**3,000/3,000 = 100.00%**	[99.87, 100.00]	33.04	33	37	—	—
training budget (400 simulations per move; the matched one-factor comparison)							
final recipe, seed 0	1,000/1,000 = 100.00%	[99.62, 100.00]	33.40	33	38	12.7	110,986
final recipe, seed 1	998/1,000 = 99.80%	[99.27, 99.95]	32.97	33	38	23.1	112,043
final recipe, seed 2	998/1,000 = 99.80%	[99.27, 99.95]	33.66	34	38	28.7	106,638
aggregate (mean, range)	99.87% (99.8–100.0)	—	33.34 ± 0.35	—	—	—	—
witnessed-peak value target (the pipeline's training deviations)							
no end discounting	312/1,000 = 31.20%	[28.40, 34.14]	43.47	41	57	12.9	32,331
episode-scalar discounting	653/1,000 = 65.30%	[62.30, 68.19]	39.94	39	47	32.6	7,808
no episode maximum	0/1,000 = 0.00%	[0.00, 0.38]	—	—	—	127.4	2
no solve margin (WIN_VALUE=1)	0/1,000 = 0.00%	[0.00, 0.38]	—	—	—	35.3	131
augmentation (AlphaGo Zero used Go's 8 board symmetries)							
no symmetry augmentation	0/1,000 = 0.00%	[0.00, 0.38]	—	—	—	never	0

5 Experiments

Protocol. The training protocol is fixed and identical across every run. We report the eval-time search budget separately (Table 4). Each run trains from scratch for 240 minutes of wall-clock self-play on a single Tesla V100S-32GB with the configuration of Section 4, running 400 search simulations per move throughout. Runs are matched on wall clock, not gradient steps. This gives the solving runs about 20% more SGD steps than the arms that never learn to solve, a difference far too small to account for the gaps in Table 3 (Appendix B).

Progress during training is monitored by a periodic probe, which runs greedy evaluation on a fixed held-out set of 512 uniformly random cubes every 150 iterations, using the same cubes for every run. The final measurement is greedy evaluation on the standard DeepCubeA test set of 1,000 cubes scrambled 1,000–10,000 times (Agostinelli et al., 2019), the benchmark the goal-anchored systems of Table 1 report on. *Greedy* evaluation fixes the action rule: root noise off, argmax action selection, and an 80-move cap (double the training budget, so an imperfect policy is not truncated mid-solve). It leaves the per-move search budget free (Table 4). We report the recipe at 1,024 simulations per move throughout, and the ablation arms at the 400-simulation training budget. The main recipe runs three seeds. Each ablation arm runs one seed, so every arm in Table 3 is a single run and the failure dynamics described below are specific to that run. That design was fixed before any results were available, under the escalation rule of Appendix B.

Main result. The recipe solves the full benchmark. At the 1,024-simulation budget we report throughout, each of the three seeds solves all 1,000 DeepCubeA test cubes (3,000/3,000, 100%; Tables 3 and 4). That matches the solve rate of the goal-anchored systems of Table 1 while training only on uniformly random cubes. Most of this performance comes from the network alone. Without search, the raw policy head solves 78.8% of the test set (per seed 86.7%/88.2%/61.6%), whereas every unlearned exploitation of the same reward at affordable budgets achieves 0% (Section 3.2). Search accounts for the rest (Table 4).

The three seeds agree in their training curves as well as in their final results (Figure 2a). Each stays at $\leq$2% on the held-out probe through the first 75 minutes. It then rises to 77–91% by minute 104/105/106 and then to a 99–100% plateau that holds for the rest of the run. We call this rise the *transition*. The transition does

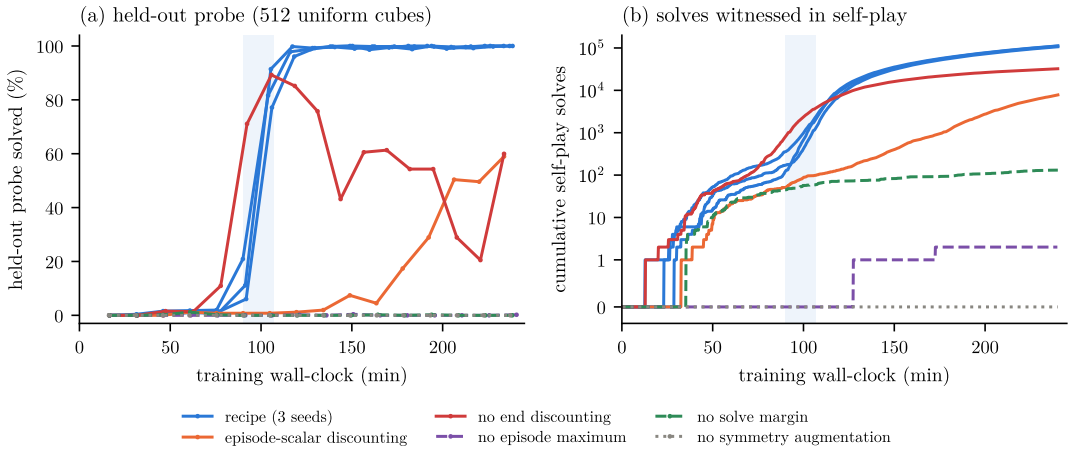


Figure 2: Training dynamics of all eight runs of Table 3. (a) Held-out probe solve rate (512 uniform cubes, every 150 iterations) against wall-clock time. The band marks the 90–107-minute window over which all three recipe seeds make the transition. (b) Cumulative self-play episodes that reach the goal (symlog). The recipe witnesses a few hundred solves in its first ninety minutes, the same order as the no-solve-margin arm accumulates in four hours without ever making that transition. The recipe's count then grows by three orders of magnitude. The no-symmetry arm never witnesses a solve.

Table 4: Solve rate on the DeepCubeA 1,000-cube test set against the eval-time search budget, with the three trained recipe networks held fixed (greedy protocol of Section 5). Column "0" is the raw policy head, argmax over the 15 moves surviving the redundancy cut, so the walk carries a one-step face memory but no tree. Three-seed mean. Mean solution length is flat at the higher budgets (≈33 half-turn moves). Seven of the sixteen swept budgets are omitted for width.

sims per move	0	32	64	128	256	400	512	1,024	2,048
solve% (3-seed)	78.8	86.0	89.9	96.1	99.7	99.9	99.9	**100.0**	**100.0**

not begin at the first witnessed solve. The first solves come much earlier (minute 12.7/23.1/28.7) and are far apart, and only 352/132/177 self-play solves have accumulated by the last probe before the transition (Figure 2b). The ablations below are compared against this reference schedule. The uniform-cube probes and the DeepCubeA set agree at the end of training, so the result is not benchmark-specific.

Ablations. The witnessed-peak value target has three components, and Table 2 lists them alongside the symmetry augmentation. Each is removed in its own arm at matched compute. End discounting also gets a second arm that softens it rather than removing it (Table 3). Every arm lands far outside the spread of the three recipe seeds, and the arms fail in two distinct ways.

Three arms never make the transition. Removing the episode maximum leaves the standard terminal outcome and only 2 solves in 153,600 episodes, and removing the symmetry augmentation yields no solves in 155,456. Removing the solve margin fails despite witnessing 131 solves, the same order as the 132–352 each recipe seed witnesses before its transition. The margin determines whether a witnessed solve reorders the value landscape. With the solve value equal to the reward ceiling, a witnessed win never dominates.

The other two learn to solve but finish well below the recipe. Flat solve labels make the transition on the recipe's schedule (probe 89% at minute 106) and then degrade where the recipe plateaus. The probe series swings between 21% and 85% over the remaining two hours, and the final evaluation measures 31.2% with much longer solutions. In this run, end discounting is necessary for stability and solution length, even though it is not needed for the transition. Episode-scalar discounting keeps the preference for short solves but drops the per-state within-episode gradient. It reaches 65.3% while still rising at the budget, so that number

measures how far it gets in 240 minutes rather than what it would converge to. Appendix B gives each arm's probe series and the one-factor control relationships among them.

Solution quality. Greedy solutions are long. In the quarter-turn metric the baselines report (Table 1), the three-seed mean is 54.4 QTM, about 2.5× DeepCubeA's 21.50-QTM mean on the same 1,000 cubes. That is 33.04 ± 0.38 HTM, because 65% of our moves are half-turns, which HTM counts once but QTM twice. All 3,000 solutions are longer than DeepCubeA's mean, and our shortest is 35 QTM. The half-turn skew and the excess length have a common cause. Half of each solution is spent inside a domino group (Section 6.1), where only ten of the eighteen moves keep the state in the group and six of those ten are half turns. The half-turn share is 48.9% before the agent enters the group and 79.5% after. Routing to the goal through an intermediate subgroup, rather than directly, is the cost of a two-phase strategy, and our agent incurs it without being explicitly trained to. For scale outside the learned literature, in half turns throughout, optimal solvers need at most 20 (Rokicki et al., 2014), human speedsolving methods average roughly 50–60, and beginner layer-by-layer methods roughly 100.

Compute. Each run takes four hours on one V100. The eight runs of Table 3, plus the eval-time sweep and ancillary jobs, total ∼34 V100-hours.

6 Analysis

A state-independent policy solves within the budget with probability below 10^{-18} (Appendix C), against $\sim 10^5$ episodes per training run, which rules out chance. Unlearned exploitation of the completion reward does not explain the trained agent's solve rate either, since no procedure we tried solves any test cube at an affordable budget (Section 3.2). McAleer et al. (2019) motivated goal-anchored generation on this basis, writing that "even a naive monte-carlo approach would not work since it will never encounter the solved state." Our measurements confirm that premise. However, our result refutes the conclusion drawn from it, namely that forward-only training must be supplied with solves generated backward from the goal. This section gives our account of what training learns that no affordable fixed procedure finds, with each claim labeled as measured or as a hypothesis.

Our account rests on two facts. First, mean completion falls monotonically with true distance out to d=11, the limit of our exact distances (Section 3.2), even though the per-state signal there is nearly uninformative. A learner that averages over many episodes can therefore detect a trend that no single-trajectory procedure can exploit. Second, the episode-max target gives every visited state its trajectory's best peak, a value that is witnessed-reachable by reversibility. Our hypothesis is that the two effects reinforce each other in the usual self-play loop until solves are witnessed. From that point, every state on a witnessed solve is valued above every non-solving return (Section 4.2), and the loop commits to solving.

Reward shaping is a real concern. The return is not potential-based (Ng et al., 1999), so no policy-invariance guarantee applies. A policy that reliably stays at a 50/54 local maximum would outscore a policy that solves fewer than 13% of cubes.[2] However, high-completion states are necessarily close to solved (completion ≥ 0.85 leaves at most eight wrong facelets). The states a reward-hacking policy would target therefore lie near the goal and offer no shortcut to an agent starting from a scrambled cube. Once solves occur, the doubled solve value makes staying at a local maximum strictly dominated. When a shallower search leaves a cube unsolved, the failures are deterministic limit cycles with peak completion 0.43–0.85, below the 50/54 traps.

Replaying the solutions shows how the policy uses the reward (Figure 3). None of the 3,000 greedy solutions from the recipe seeds is monotone non-decreasing in completion. Of the played moves, 34.1% strictly decrease completion, and 57.1% of solutions fall below their own starting completion before reaching the goal. The policy raises r on 53.6% of moves, against 25.0% for a uniformly random legal move at the same states, so it is not indifferent to the reward. Mean completion is nonetheless only 0.57 at 90% of the way through a solution, so the policy is far from simply climbing the reward.

[2]Staying at $r = 50/54$ yields a deterministic return of 50/54. A policy solving a fraction p of episodes with mean completion h_f on the rest returns $2p + h_f(1-p)$, so staying wins whenever $p < (50/54 - h_f)/(2 - h_f)$, which is 13% at $h_f \approx 0.77$.

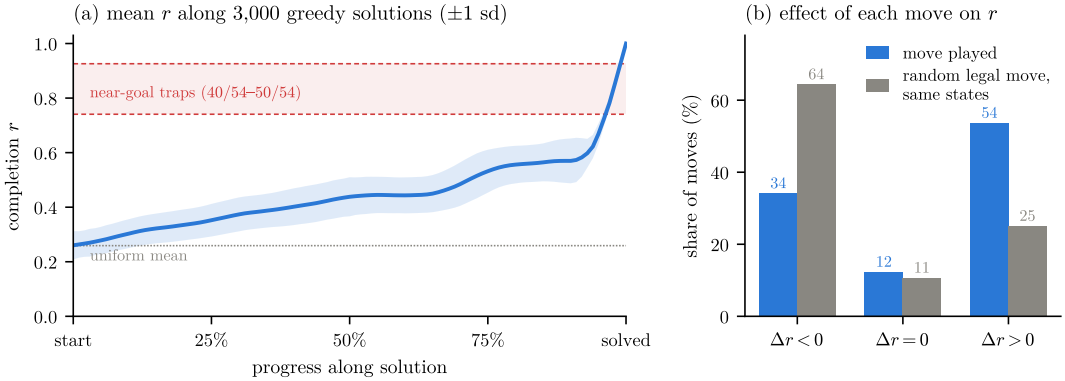


Figure 3: The learned policy is not a hill-climber on r. (a) Mean completion along the 3,000 greedy solutions of the three recipe seeds, replayed from their start states, against normalized solution progress. The route to the goal stays below the entire near-goal trap band until the last 4% of the solution. Completion r first exceeds the nearest trap value 40/54 at 96% of the way through, and never exceeds the deepest trap value 50/54 before the solved state itself. (b) Effect of each played move on r, against the matched control of a uniformly random legal move at the same visited states.

Part of the final rise is structural rather than behavioral. States near the goal necessarily score high, and every goal-adjacent state scores exactly 42/54, so the last few moves must pass through states that 330 near-goal maxima outrank (Section 3.2). The non-monotonicity and the rate of completion-decreasing moves, however, are properties of the policy. Go-Explore overcomes deceptive rewards with an explicit archive of waypoints (Ecoffet et al., 2021). Here, whatever overcomes the traps is implicit in search plus the episode-max target.

6.1 Domino reduction

The agent's solutions end with a long run of moves inside a *domino group*. For one of the three axes this is $H = \langle U, D, L^2, R^2, F^2, B^2 \rangle$, generated by all turns of the top and bottom faces together with only half turns of the remaining four. It is the target of the second stage of Thistlethwaite's algorithm and of the first phase of Kociemba's two-phase solver (Thistlethwaite, 1981; Kociemba, 1992). A uniformly random cube lies in one with probability 1.35×10^{-9}. Over the 3,000 cubes the three recipe seeds solve, the final 16.74 ± 2.68 moves lie in a single such group, half the solution, against 2.53 for a length-matched null that also ends solved. Part of this effect comes from the reward rather than from the agent. The unlearned beam of Section 3.2 climbs the same reward without learning and already has a 10.36 ± 5.85-move suffix, because completion ranks the subgroup 3.5 standard deviations of the uniform spread above a typical state. Training makes the suffix longer and more reliable. The beam's suffix reaches eight moves on 68% of cubes and is one move in the worst case. In contrast, all but one of the 9,330 learned solutions we recorded reach eight, at every eval-time budget and in both ablation arms that learn to solve. The deepest traps of Section 3.2 are this same structure seen from the goal. The 42 states at $d = 6$ scoring $r = 50/54$ are domino states in all three axes at once. The reward's worst near-goal inversion is therefore exactly where an agent seeking the domino group would arrive, and only the solve margin keeps the agent from stopping there (Section 4.2). This is a measured property of the trajectories, not evidence that the network represents H. Appendix E gives the membership predicate, the null, the per-configuration table, and the rest of the measurement.

7 Limitations

Solutions are long, about 2.5× DeepCubeA's mean on the same cubes, and deeper eval-time search does not shorten them. Goal-anchored methods (Table 1) remain the right tools when near-optimal solutions are the objective. Our hardness evidence concerns affordable search rather than information content, since a beam on r given a whole training run's node budget per cube solves all 50 benchmark cubes we ran it on (Section 3.2). The domino account is correlational (Section 6.1). We have not tested whether the phenomenon generalizes

beyond this case, in which a coarse subgroup happens to coincide with a high-completion region. All of our evidence comes from the 3×3×3 cube with one pattern-matching reward. Larger cubes pose a harder version of the same landscape, and we have not attempted them. Finally, we use a redundancy cut and a 48-element symmetry augmentation (Table 2).

8 Conclusion

We showed that goal-anchored training data, which every prior learned solver of the full cube relies on, is not necessary. A standard AlphaZero pipeline, rewarded only by the naive pattern match against the goal plus the solve event, solves the entire DeepCubeA benchmark after four hours on one GPU. Its solutions are about 2.5× DeepCubeA's mean length. Apart from a symmetry augmentation, the recipe changes AlphaZero in one place only, the witnessed-peak value target. The trained agent converges on a classical decomposition of the problem rather than a better exploitation of the reward.

Goal-anchored data may be less necessary than its uniform use across the cube literature suggests, and its necessity is worth testing before it is assumed. How far forward-only self-play scales as the landscape becomes harder remains open, and the 4×4×4 cube is the natural next test.

Reproducibility Statement

The implementation, configuration, analysis scripts, and raw landscape data are released at `https://github.com/Ramora0/cubric`, together with the raw measurements behind every number in Section 5: per-cube final-eval dumps, the full periodic-eval series, and per-iteration metric exports for all eight runs. One command reproduces each arm. The reported runs were executed from a frozen pre-release snapshot of that code, which we verified arm by arm to be semantically identical to the release apart from flag spellings, and the artifact records that comparison. Parity tests pin the production search kernel as bit-identical to a plain-numpy reference (Appendix F). Full hyperparameters are in Appendix A.

References

Forest Agostinelli, Stephen McAleer, Alexander Shmakov, and Pierre Baldi. Solving the Rubik's cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1(8):356–363, 2019.

Alessandro Barro. Node classification and search on the Rubik's cube graph with GNNs. *arXiv preprint arXiv:2501.18580*, 2025.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, pp. 41–48, 2009.

Charles Blundell, Benigno Uria, Alexander Pritzel, Yazhe Li, Avraham Ruderman, Joel Z. Leibo, Jack Rae, Daan Wierstra, and Demis Hassabis. Model-free episodic control. *arXiv preprint arXiv:1606.04460*, 2016.

Robert Brunetto and Otakar Trunda. Deep heuristic-learning in the Rubik's cube domain: An experimental evaluation. In *Proceedings of the Conference on Theory and Practice of Information Technologies (ITAT)*, volume 1885 of *CEUR Workshop Proceedings*, pp. 57–64, 2017.

Mustafa E. Chasmai. CubeTR: Learning to solve the Rubik's cube using transformers. *arXiv preprint arXiv:2111.06036*, 2021.

Alexander Chervov, Kirill Khoruzhii, Nikita Bukhal, Jalal Naghiyev, Vladislav Zamkovoy, Ivan Koltsov, Lyudmila Cheldieva, Arsenii Sychev, Arsenii Lenin, Mark Obozov, Egor Urvanov, and Alexey Romanov. A machine learning approach that beats Rubik's cubes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

Alexander Chervov, Mark Obozov, Anton Soibelman, et al. CayleyPy RL: Pathfinding and reinforcement learning on Cayley graphs. *arXiv preprint arXiv:2502.18663*, 2025b.

Sebastiano Corli, Lorenzo Moro, Davide E. Galli, and Enrico Prati. Solving Rubik's cube via quantum mechanics and deep reinforcement learning. *Journal of Physics A: Mathematical and Theoretical*, 54(42): 425302, 2021.

Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.

Sai Krishna Gottipati, Yashaswi Pathak, Rohan Nuttall, Sahir, Raviteja Chunduru, Ahmed Touati, Sriram Ganapathi Subramanian, Matthew E. Taylor, and Sarath Chandar. Maximum reward formulation in reinforcement learning. *arXiv preprint arXiv:2010.03744*, 2020.

Terry Jones and Stephanie Forrest. Fitness distance correlation as a measure of problem difficulty for genetic algorithms. In *Proceedings of the 6th International Conference on Genetic Algorithms (ICGA)*, pp. 184–192, 1995.

Herbert Kociemba. The two-phase algorithm, 1992. https://kociemba.org/cube.htm.

Wolfgang Konen. Towards learning Rubik's cube with N-tuple-based reinforcement learning. *arXiv preprint arXiv:2301.12167*, 2023.

Alexandre Laterre, Yunguan Fu, Mohamed Khalil Jabri, Alain-Sam Cohen, David Kas, Karl Hajjar, Torbjørn S. Dahl, Amine Kerkeni, and Karim Beguir. Ranked reward: Enabling self-play reinforcement learning for combinatorial optimization. In *NeurIPS Deep Reinforcement Learning Workshop*, 2018.

Yicheng Lin and Siyu Liang. Solving Rubik's cube without tricky sampling. *arXiv preprint arXiv:2411.19583*, 2024.

Zichuan Lin, Tianqi Zhao, Guangwen Yang, and Lintao Zhang. Episodic memory deep Q-networks. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2433–2439, 2018.

Stephen McAleer, Forest Agostinelli, Alexander Shmakov, and Pierre Baldi. Solving the Rubik's cube without human knowledge. *arXiv preprint arXiv:1805.07470*, 2018.

Stephen McAleer, Forest Agostinelli, Alexander Shmakov, and Pierre Baldi. Solving the Rubik's cube with approximate policy iteration. In *International Conference on Learning Representations (ICLR)*, 2019.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278–287, 1999.

Gabriel Poesia, WenXin Dong, and Noah D. Goodman. Contrastive reinforcement learning of symbolic reasoning domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge. The diameter of the Rubik's cube group is twenty. *SIAM Review*, 56(4):645–670, 2014.

Christopher D. Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

Jason Rute. Solving the puzzle cube with deep reinforcement learning and tree search. GitHub repository, 2018. URL https://github.com/jasonrute/puzzle_cube.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

Kyo Takano. Self-supervision is all you need for solving Rubik's cube. *Transactions on Machine Learning Research (TMLR)*, 2023.

Morwen B. Thistlethwaite. The 45–52 move strategy. Unpublished manuscript, 1981. Transcribed at `https://www.jaapsch.net/puzzles/thistle.htm`.

Grigorii Veviurko, Wendelin Böhmer, and Mathijs de Weerdt. To the max: Reinventing reward in reinforcement learning. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pp. 49455–49470, 2024.

Alicja Ziarko, Michał Bortkiewicz, Michał Zawalski, Benjamin Eysenbach, and Piotr Miłoś. Contrastive representations for temporal reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

A Full Hyperparameters

Table 5 transcribes the configuration logged by every run of Section 5, and each run's released eval dump records it verbatim. Every entry matches the released `Config` defaults, so the released defaults are exactly this recipe. A run launched with no overrides therefore reproduces the main recipe, and each ablation arm is a command-line override of these defaults (Table 3). The recipe has no per-seed or per-stage tuning. Experimental flags not listed default to off.

Table 5: Hyperparameters of the recipe evaluated in Section 5.

Network (DeepCubeA trunk)		*Search*	
input	54×6 one-hot (324)	simulations per move	400
layer 1 / layer 2	5000 / 1000	c_{puct}	1.0
residual blocks	4×1000	Dirichlet α / ε	0.3 / 0.25
policy head	18 logits	value backup	mean ($Q = W/N$)
value head	scalar	leaf wave / virtual loss	16 / 1.0
		root action	argmax visits
Episode and targets		*Optimization and replay*	
start distribution	uniform solvable	optimizer	Adam
move budget (train / eval)	40 / 80	lr / weight decay	$5{\times}10^{-4}$ / 10^{-4}
redundancy cut	on	batch size	512
value target (unsolved)	episode-max r	replay	16 rounds, 2×10^5 cap
value target (solved)	$2 - 0.02\,m$ (m: moves to solve)	symmetry augmentation	48-group, per draw
games per iteration	64	sample reuse	48 draws/sample

B Protocol, Provenance, and Per-Arm Dynamics

Comparability of matched-compute arms. A run that solves ends its episodes early and so completes more iterations, 2,871–2,945 for the three recipe seeds against 2,400–2,429 for the arms that never make the transition. This amounts to about 20% more SGD steps. A second asymmetry favors us. DeepCubeA and its successors train on reverse scrambles of depth 1–30 and test on effectively uniform cubes. Our training distribution equals our evaluation distribution, so we avoid a generalization gap that the goal-anchored baselines must cross. This is the counterpart to the curriculum that their data generation gives them and ours forgoes. The Wilson intervals of Table 3 cover sampling of the 1,000 test cubes only. Run-to-run variation is much larger than this sampling error when solve rates are below saturation, where the same three recipe networks score 86.7/88.2/61.6 with no search.

Seed escalation and run provenance. Each ablation arm runs one seed under an escalation rule fixed before any results were available. An arm whose final result lands within the spread of the three recipe seeds receives two more seeds. The rule was never triggered, and in retrospect it was unlikely to be, since the recipe's spread is measured at saturation (99.80–100.00). That range is so narrow that only an arm fully reproducing the recipe could land in it. Seeds 1 and 2 of four arms were launched and cancelled roughly ten minutes in, before any probe or checkpoint, under that same decision made before any results (`experiments/logs/SNAPSHOT_PROVENANCE.txt`). The single-seed design is therefore not a post-hoc selection.

Per-arm dynamics. *No end discounting* (solves regress a flat WIN_VALUE = 2) is the only removal arm that learns to solve. Its probe solve rate swings by tens of points between successive probes as it declines, and its solutions have mean length 43.5 against the recipe's 33.4 (p90 57 against 38). *Episode-scalar discounting* (every state of a solved episode regresses one scalar WIN_VALUE $- 0.02\,L$ for solve length L) scores 1% on the probe at minute 120 and 59% at minute 234, its last probe. Retaining the length preference recovers about half of the drop caused by flat labels (31%→65%). This is consistent with the missing ingredient being the per-state gradient rather than the preference for shorter solves. We did not run it longer. *No episode maximum*: no probe exceeds 2/512, and self-play mean completion plateaus near 0.56. Only 5 of 153,600 episodes ever reach 90% completion, two of them solves (the first at minute 127), which is two orders of magnitude too few to start a bootstrap. *No solve margin*: no probe exceeds 6/512 and only four of sixteen are nonzero, even though early self-play is indistinguishable from the recipe's (mean completion ∼0.69 at minute 15 against the recipe's 0.70). This arm runs with zero end discounting. Under WIN_VALUE = 1, a decayed solve label would fall below typical mid-episode completion labels, which would penalize solving and weaken the arm beyond the single factor under test. Its matched one-factor control is therefore the no-discounting arm (WIN_VALUE 2 against 1, all else equal), which reaches 89% on the probe. *No symmetry augmentation*: 0/512 on every probe, and none of 155,456 episodes reaches the goal, so the arm never witnessed a solve from which the transition could start.

C Probability of Solving by Chance

Identify states with elements of the cube group ($|G| = 4.325 \times 10^{19}$). A uniformly random start is uniform over G, and applying any fixed move is multiplication by a fixed group element, which preserves uniformity. Hence, under any policy whose action choices are independent of the state (the uniform-random policy in particular), every state an episode visits is marginally uniform. A 40-move episode visits 41 states, so by the union bound

$$\Pr[\text{chance solve}] \;\le\; \frac{41}{4.325 \times 10^{19}} \;<\; 10^{-18}.$$

A full training run generates on the order of 10^5 self-play episodes, thirteen orders of magnitude short of the $\sim 10^{18}$ episodes per expected chance solve. The redundancy cut does not change the order of magnitude.

D Reward-Landscape Probes

Detail deferred from Section 3.2. A face turn changes r by at most 12/54. The turn moves 20 facelets, but the 8 on the turned face only permute within that face, which leaves its correct count unchanged. Only the 12 that move between adjacent faces can change r. The full beam sweep solves 0/50 at widths 10^3 and 10^4. It reaches a 25% solve rate at 2.7×10^7 expansions, 90% at 2.1×10^8, and 100% at 9.2×10^8, the last at the expansion-matched width 3.75×10^7 with solutions of mean length 25.5. These are order statistics of the 50-cube sample and would be higher over 1,000 cubes, so the argument depends on the orders of magnitude, not the individual values. Width is the only setting that helps, for the reason given at the end of this appendix. The nearest strict local maxima lie three moves from solved ($r = 40/54$, e.g. `R2 F2 R2`). Intermediate strata are reported with the raw data, where pairwise ordering accuracy between adjacent distance strata falls from 1.00 at d=1 against d=2 to 0.53 by d=9 against d=10.

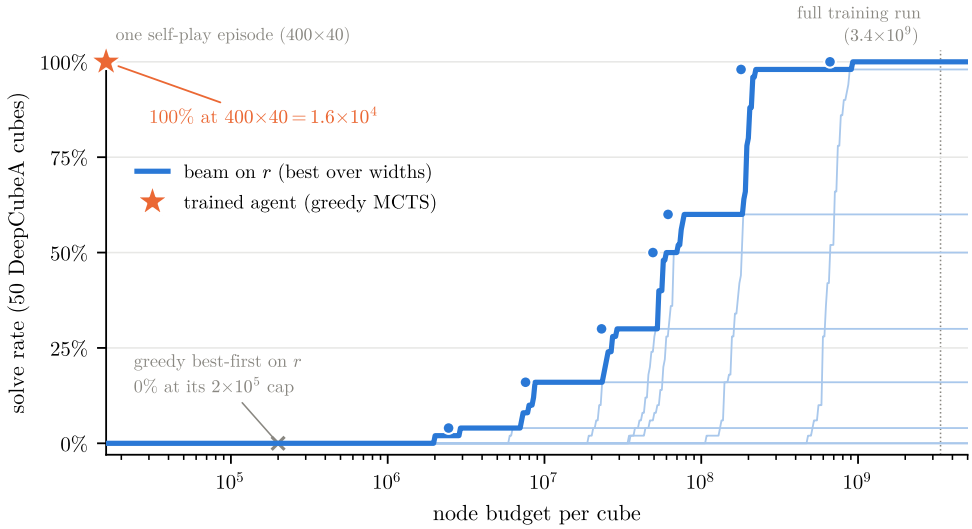


Figure 4: The cost of exploiting the completion reward without learning. Solve rate of beam search on r against node expansions per cube (50 DeepCubeA cubes). Faint lines are the individual beam widths, and the heavy line is their upper envelope. The reward can be exploited at 10^7 to 10^9 expansions per cube. One self-play episode spends 1.6×10^4.

This appendix summarizes the measurement protocol behind Section 3.2. The complete raw data accompany the code (`analysis/landscape/DATA.md`, generated directly from the result files in `analysis/landscape/results/`, with per-measure seeds and sample sizes embedded).

Ground-truth distances. Exact distances are computed from a breadth-first table of all 8,240,087 states within radius 6 of the goal. Its per-level counts equal the published half-turn-metric census (Rokicki et al., 2014), validating the move model end-to-end. Meet-in-the-middle forward expansion extends exactness to $d \leq 11$. Beyond that we use a two-phase solver upper bound, minimized over the 48 symmetry conjugates of each state. On 300 states with known exact distance, the bound is exact in 289 cases (mean gap +0.12). Because the bound is noisier in the bulk of the state space, the flat per-stratum completion means (mean $r = 0.252$–0.261 for every $\hat{d} \in [17, 21]$) support the conclusion that r is uninformative independently of the correlation estimate. Scramble depth is an accurate distance label only up to $k \approx 11$ ($\Pr[d{=}k]$ falls from 1.0 at $k \leq 2$ to 0.67 at $k = 11$), so all quantitative claims are stated against measured distances rather than scramble depth.

Measures. We report these measures in addition to the fitness–distance analysis and search comparison of Section 3.2. Random-walk autocorrelation of r has correlation length $\tau = 2.91$ moves ($\rho(1) = 0.709$). Because 18.7% of steps leave r unchanged, τ is if anything an overestimate of smoothness. The reward decorrelates within three moves, against a solution scale of ∼18 moves. At a uniformly random state, 40.6% of the 18 moves increase r and 18.7% leave it unchanged. In the bulk of the state space, r can therefore be climbed everywhere, but the climb is nearly neutral with respect to distance. Where exact distances permit a direct check (radius-6 table), the steepest-completion move reduces true distance in only 38%/28%/20% of states at d=4/5/6, against a 6% base rate. The state-level r_{FDC}=0.87 within $d \leq 5$ therefore does not carry over to individual moves. Plateau regions of equal r percolate, and all 40 flood fills from uniform seeds exceeded a 2×10^5-state cap. Strict local maxima occupy 0.38% of the space by volume, yet steepest-ascent failures end at one roughly twice as often as they exhaust a plateau. The deepest near-goal traps are the 42 states at $d = 6$ with $r = 50/54$, which have two disjoint pairs of exchanged edges and all corners solved. In best-first runs that exhaust their budget, the maximum r ever popped lies in $[0.852, 0.944]$ (median 0.926). The frontier reaches states three to eight facelets from solved and still never finds the goal.

Table 6: Solve rate of unlearned procedures exploiting the completion reward on uniformly random cubes (Section 3.2). The budget column is a cap for the best-first row and the mean actually spent for the beam rows. The entry *n/a* marks a procedure to which no node budget applies. The four cheap procedures do solve cubes scrambled a few moves from the goal, consistent with the near-goal correlation in Figure 1. Those per-depth rates are released with the code. The beam rows run on the first 50 cubes of the DeepCubeA test set, the set and distribution the trained agent is scored on. Each cube is allotted the node budget of an entire training run.

	n	solve rate	budget (node exp./cube)
steepest ascent on r	1,000	0.0%	n/a
greedy, radius-3 lookahead	200	0.0%	n/a
greedy, radius-5 lookahead	60	0.0%	n/a
greedy best-first on r	50	0.0%	2×10^5
beam on r, width 2.5×10^6	50	60.0%	1.1×10^8
beam on r, width 3.75×10^7	50	**100.0%**	6.8×10^8

Cost units. Every ratio quoted in Section 3.2 counts node expansions, the unit in which an episode's own budget is defined. It is not a common cost unit. A beam expansion evaluates r on its 15 children, which takes 810 facelet comparisons, while an agent simulation is a forward pass through the trunk of Section 4, about 1.5×10^7 multiply-accumulates. Measured in arithmetic operations, the two comparisons of Section 3.2 give different results. The search-free policy head still costs less, since its 55 forward passes take $\approx 8 \times 10^8$ multiply-accumulates against the $\approx 1.6 \times 10^{11}$ comparisons the beam spends to reach the same 78.8%. Against the 1.6×10^4 simulations of a searched episode, however, the beam is the cheaper procedure. We use the beam to characterize the reward, not to rank solvers by compute.

The training-matched beam control. The beam rows of Table 6 and Figure 4 run on the first 50 cubes of the DeepCubeA test set, expanding the same 15 non-redundant children per node the agent's search uses and collapsing duplicate states within each level by 64-bit Zobrist hash. A solve is counted the moment the goal is generated, the same convention as the best-first row above. Every returned solution is replayed through the move engine and verified. The reference training cost is 2,945 iterations $\times$ 64 games $\times \leq 40$ moves $\times$ 400 simulations $\leq 3.02 \times 10^9$ node expansions, plus $\approx 3.4 \times 10^8$ in periodic and final evaluation. Two widths match that budget under the two readings of "expansion". Width 2.5×10^6 matches guidance-function calls (3.0×10^9 r-evaluations against the training run's 3.0×10^9 network forwards) and solves 30/50. These calls are matched in count, not in arithmetic, since an r-evaluation is a 54-facelet comparison and a network forward is a full trunk pass. Width 3.75×10^7 matches node pops ($\approx 2.9 \times 10^9$) and solves 50/50, spending 6.79×10^8 expansions per cube on average. Both matched arms run to depth 80, the agent's eval move budget. The budget curve of Figure 4 adds a separate seven-width sweep, run to depth 50, from 10^3 to 8×10^6 (0/50 solved at 10^3 and 10^4, 49/50 at 8×10^6). The figure's envelope is taken over all nine widths and so mixes the two depths. This can only cause the depth-50 runs to miss solutions that the depth-80 arms would find. Each run's solve rate against budget is read from the per-cube expansions-to-solve, so one run per width covers every budget. The envelope assumes an oracle choice of width per budget, which favors the beam, as does counting a solve at generation rather than expansion.

Where the narrower beam fails, it stops at a level beyond which the reward gives no signal. All 20 of its unsolved cubes stall with best-reached $r = 51/54$ exactly, and 51/54 is the maximum of r over non-solved states. A value of 53/54 is impossible by color counting, and 52/54 would require a flipped edge, a corner twisted with one sticker fixed, or a lone edge transposition, each of which is impossible. The 51/54 level is attained by a pure three-cycle of top-layer edges in which each displaced edge keeps its top-face sticker. Its 11-move path to the solve gives up 30 facelets (r: $0.944 \to 0.389$) and passes through states that the reward ranks below almost the entire beam frontier. The guidance-matched arm generated $\sim 8 \times 10^{10}$ child states without once observing 52/54 or 53/54, and the 0.944 ceiling of the budget-exhausted best-first runs above is this same 51/54 level.

E Domino-Reduction Probe

This appendix gives the protocol behind Section 6.1 and Table 7. The script and its raw output are released with the code (`experiments/domino_probe.py`, `experiments/logs/domino_reduction.json`).

Table 7: Domino reduction in stored solutions. *Suffix* is the length of the terminal run of moves confined to one domino group, maximized over the three axes. *Share* is that run as a fraction of the solution. The matched null is the inverse of a random scramble of the same length, which also ends solved, measured identically. The beam row is the expansion-matched unlearned beam of Section 3.2. It measures what the reward alone gives any hill-climbing search, and the amount by which the learned rows exceed it is attributable to training. A suffix of 1 is the floor of the statistic, since the solved state lies in all three groups. Protocol in Appendix E.

		domino suffix (half turns)				
Solutions	n	mean	med.	min	share	suffix ≥ 8
recipe, raw policy (0 sims)	2,365	21.40 ± 4.29	21	11	45.1%	**100%**
recipe, 1,024 sims (mainline)	3,000	16.74 ± 2.68	17	7	50.8%	**99.97%**
recipe, 1,600 sims	3,000	16.85 ± 2.70	17	8	51.1%	**100%**
episode-scalar discounting, 400 sims	653	17.49 ± 3.69	17	8	44.1%	**100%**
no end discounting, 400 sims	312	16.15 ± 3.02	16	8	38.5%	**100%**
unlearned beam on r (no learning)	50	10.36 ± 5.85	9.5	1	40.0%	68%
matched null (recipe, 1,024 sims)	3,000	2.53	2	1	7.7%	2.4%
matched null (beam)	50	2.28	2	1	9.1%	4.0%

Membership predicate. A state is in the domino group $H_a = \langle A, A', B, B', (\text{other four faces})^2 \rangle$ of axis a (faces A, B opposite on a) iff every corner is oriented about a (its a-colored sticker faces one of the two a-normal directions), every edge is oriented, and the four edges whose two colors both avoid a sit in the middle slice of a. The predicate is read directly off the facelet lattice of the released move engine, not off a separate cubie model. It is validated three ways. The solved state satisfies it for all three axes. None of 300 uniformly random states satisfies it for any axis. For each axis, it is closed under that axis's ten generators and falsified by each of the other eight moves from a state inside the group.

Measurement. Each stored solution is replayed backward from the solved state through the inverse move sequence, giving the states $s_0, \ldots, s_L$. For each axis we find the largest t with $s_k \in H_a$ for all $k \geq t$ and record $L - t$. The reported suffix is the maximum over the three axes, and the axis attaining it is the one tabulated. Because s, $sm \in H_a$ implies $m \in H_a$, the suffix equals the number of trailing moves drawn from that axis's ten domino generators, which the script asserts independently. Re-entries, earlier visits to the same group that the policy left again, are counted over $k < t$. There are none in 3,000 at the mainline budget and 2 in 2,365 for the raw policy.

Null. The matched control is the inverse of a uniformly random scramble of the same length as the solution it is matched to, subject only to the redundancy cut, so it is a move sequence of identical length that also terminates at the solved state, and it is measured by the identical procedure. Its suffix distribution is the chance level of the statistic under "ends solved, otherwise unstructured".

No-learning control. The beam row of Table 7 is the 50 verified solutions of the expansion-matched beam of Appendix D, measured by the same code path with its own length-matched null. It holds the reward and the test cubes fixed and removes learning entirely, so the gap between it and the learned rows is the part of the routing that training is responsible for. The gap between it and the null is the part the reward affords on its own.

Reward statistics. The 3.5-standard-deviation gap of Section 6.1 compares subgroup completion means over 20,000 samples per axis, each a 500-move random walk in that axis's own ten generators from the solved state (a mixing sampler over H_a, not a closed-form uniform one). The uniform baseline is the same

construction over all 18 moves and recovers the $r = 0.259 \pm 0.052$ of Section 3.2. Mean r by exact distance is computed over the full radius-6 table.

F Implementation Details

Caveats of the episode-maximum target. Walking back to a peak seen k moves earlier costs k of the 40-move budget, of which only $40 - t$ remains at step t, so a label can name a peak the episode could no longer have collected. In addition, the encoding carries no step index, so a state visited at two points of an episode receives two different labels. The target is therefore a function of the trajectory, not of the state alone, and v_θ fits their mixture.

Selection rule. Selection is the PUCT rule of Rosin (2011) as AlphaZero uses it, with two conventions worth stating. The virtual visits of leaves still in flight within a wave enter the exploration term's visit counts, and the +1 retained under its root keeps the prior active on the first descent through an unexpanded node, where every Q ties at the first-play-urgency value. An unvisited action takes the node's own value as its Q (first-play urgency, no reduction). Priors are masked to the legal children and renormalized at every expansion (uniform if the surviving mass falls below 10^{-8}), root Dirichlet noise is drawn over the 15 legal root moves rather than 18, and the value head is a bare linear unit with no squashing, as targets above 1 require. Which three moves the redundancy cut zeroes depends on the previous move, which the encoding does not carry, so the head fits $P(a \mid s)$ against labels drawn from $P(a \mid s, \text{last face})$. The mismatch is small but it is why the mask is reapplied at inference.

Search kernel. The per-game trees live as a forest in flat preallocated arrays and are walked by nogil compiled kernels spread over two threads. Leaf evaluations from all trees merge into single batched network calls. Each tree gathers 16 leaves per wave under a virtual loss of 1.0 before one forward pass. Wave width 1 recovers exact one-leaf lockstep. The virtual visits enter the exploration term only and not Q, and backups are deferred to the end of a wave, so the waved search is a leaf-parallel approximation of sequential PUCT rather than exactly equal to it. A root child that is the solved state is promoted to a one-hot visit distribution, so a proven win cannot lose the argmax to visit noise. At those states, one move from a solve, the one-hot is also what self-play records as the policy target.

Parity tests. A plain-numpy reference implementation mirrors the production kernel, with scores computed in float64 so tie-breaking matches, and tests assert exactly equal root visit counts between the two across four regimes: root noise on and off, crossed with one and two walker threads.

Wall-clock. One iteration (64 self-play games plus $\max(1, \text{round}(48\, n_{\text{fresh}}/512)) \approx 230$ SGD steps, for n_{fresh} states newly stored) averages $\sim$5 s on the V100S, so a 240-minute run completes the 2,400–2,945 iterations of Section 5. In the arms that never make the transition, almost every episode exhausts the full 40-move budget. The final 1,000-cube greedy evaluation takes $\sim$31 s.